

ZSWQ 总体部预研创新类外协课题 开题论证报告

项目名称：基于多要素融合的算网资源智能调度算法

成果形式：算法模型类

起止年月：2025 年 9 月-2026 年 12 月

承担单位：北京邮电大学

项目负责人：谢人超

日 期：2025 年 11 月

目录

1. 概述	1
2. 研究目标、研究内容与技术指标	1
2.1. 研究目标	1
2.2. 技术指标	错误！未定义书签。
2.2.1. 功能要求	错误！未定义书签。
2.2.2. 性能要求	错误！未定义书签。
2.3. 研究内容	2
3. 研究方案	2
3.1. 设计思路	3
3.2. 设计方案	6
3.2.1. 系统模型	6
3.2.2. 基于深度强化学习的节点簇内集中式任务协同调度机制	13
3.2.3. 基于多节点联合调度的节点簇间调度机制	18
3.3. 通用质量特性要求	20
3.4. 评估标准	21
3.5. 验证方案	22
3.5.1. 验证要求	22
3.5.2. 训练样本仿真生成	23
3.5.3. 验证项目	23
4. 技术指标满足情况分析	24
5. 研究进度、预期研究成果及应用方向	25
5.1. 研究进度	25
5.2. 预期研究成果	25
5.3. 应用方向分析	25
5.3.1. 典型任务 1：时敏性任务	25
5.3.2. 典型任务 2：优先级任务	26
6. 保障条件及措施	错误！未定义书签。
6.1. 研发团队保障条件	错误！未定义书签。
6.2. 保密措施	错误！未定义书签。
6.3. 质量保障措施	错误！未定义书签。
7. 承担研究任务主要人员概况	30
8. 技术风险分析	31
9. 经费概算	32

1. 概述

随着现代战争向信息化、智能化方向的不断演进，作战场景呈现出动态性和复杂性高度交织的特点。战场环境瞬息万变，多源异构数据的大规模涌入对实时决策提出了前所未有的挑战。指挥与作战的成效不仅依赖于态势的准确感知，还取决于对数据处理和分析结果的快速反馈。这些任务往往需要处理大规模数据，进行精密计算，其结果直接影响后续决策与任务执行的质量和速度。然而，这些任务对计算性能和资源提出了极高的要求，尤其在实时性方面，现有系统难以满足其需求，成为战场指挥系统的重要瓶颈。除此之外，如何在大规模算力调度场景下进行合理调度也成为了作战系统面对的核心问题。由于作战系统中包含的作战车辆通常较多，指挥车辆集中式调度往往难以支撑同时对所有作战车辆进行调度，且需要较长的时间进行算力决策。

本方案的核心创新在于提出了兼具任务优先级度量与时敏任务处理的大算力任务调度算法，算法通过分布式节点间的资源协同，整合多节点算力，实现对高算力任务的动态分配与并行处理，突破了单节点算力限制带来的瓶颈。协同调度算法使得各任务可以在不同节点上并行执行，同时减少任务间的通信开销。动态调度机制在节点状态和环境变化时合理调整调度方案，确保任务执行效率。

算法对节点进行簇间分布式和簇内集中式调度，通过优先级感知与深度强化学习优化算法等实现节点间任务调度。在作战环境下，算法充分考虑任务的优先级，采用马尔可夫决策过程与深度强化学习生成调度策略，确保任务的及时执行并保障节点资源利用效率。同时，在任务处理过程中，算法定时检测任务的执行情况，将执行失败的任务退回就绪队列重新进行调度处理，以减少任务处理失败对于整体任务的影响，从而提高算法整体效率。

2. 研究目标、研究内容与技术指标

2.1. 研究目标

本研究的总体目标是设计一个高效、智能、动态的作战任务调度方案，旨在应对现代信息化战争中复杂多变的战场环境以及不同类型任务的调度需求。本研究将探索并实现一种集成化的任务调度框架，基于战场实际需求优化资源配置、提高调度效率，确保在动态战场环境中任务的及时执行。本研究的主要目标包括节点实时调度能力的优化。

2.2. 技术指标

2.2.1. 功能要求

a) 能够通过监控感知网络中遍布不同的算力资源，自动编排算力资源规划策略。

2.2.2. 性能要求

- a) 支持 5 种场景驱动下的网络资源调配能力；
- b) 算网资源一体化调度策略生成时间不大于 5s

2.3. 研究内容

在现代信息化战争中，作战场景的动态性和复杂性对任务调度与资源分配提出了极高的要求。节点资源的异构性、任务负载的时变性以及战场环境的强干扰性，使得传统调度方法难以满足现代作战需求。为此，本研究旨在设计一套智能化、高效化的任务调度系统，通过多维度智能模型的协同优化，动态适配复杂战场环境下的任务与资源关系，增强资源调度的鲁棒性与任务执行效率。具体研究内容包括：

（1）研究内容 1：基于多节点联合决策机制的节点簇间调度算法设计

本研究设计了一种基于多节点联合决策机制的分布式节点任务调度机制，通过多智能体决策生成任务调度策略。通过强化学习算法的高效决策性能，能够选出当前最适合执行相关任务的节点簇，从而在优化各节点簇负载均衡的同时提高任务调度的响应速度。

（2）研究内容 2：基于双延迟深度确定性策略梯度（TD3）与任务优先级的簇内算网资源调度算法设计

本研究提出一种基于 TD3 算法和任务优先级排序的算网资源调度方案，针对现代信息化战争中复杂多变的战场环境，以及突发性任务的调度需求，设计高效、智能、动态的作战任务调度系统。根据优先级对任务进行排序，优先调度高优先级任务，确保关键任务的及时执行。同时，通过构建马尔可夫决策过程（MDP），利用 TD3 算法的双 Q 网络结构、目标网络与延迟策略更新、目标策略平滑正则化等机制，优化任务调度策略，提高系统在复杂动态环境中的适应性和稳定性，确保任务的及时执行。

3. 研究方案

3.1. 设计思路

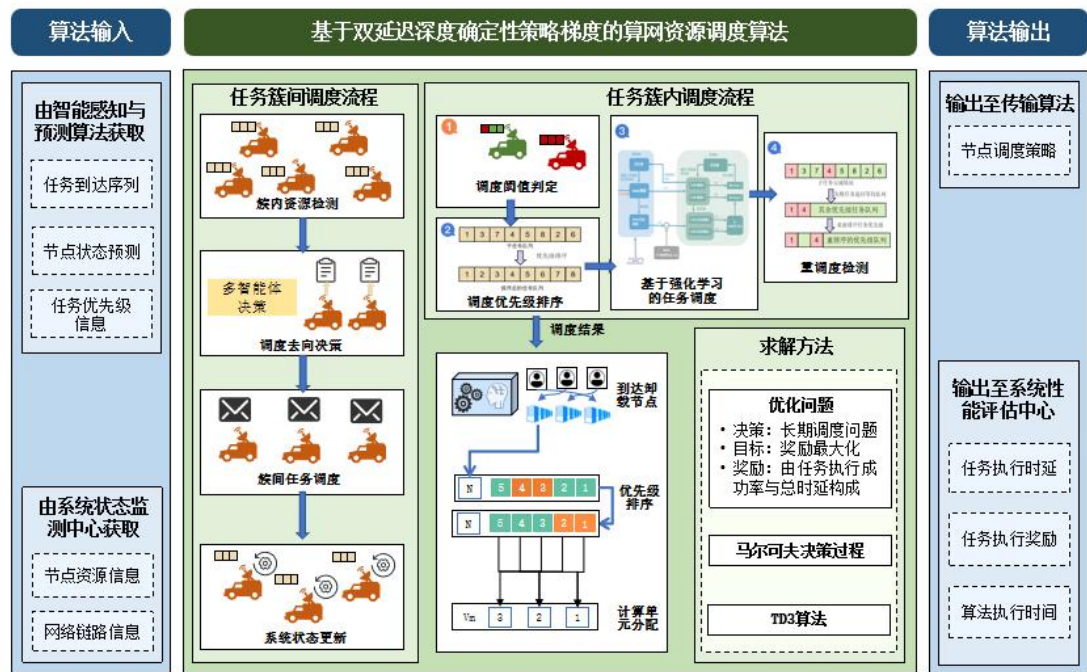


图 1 基于多要素融合的算网资源智能调度算法设计思路

算法整体设计思路如图 1 所示，基于多要素融合的算网资源智能调度算法，以处理作战场景下的突发任务与多类任务需求为方案场景，以任务总时延为优化目标，生成任务调度策略，对计算任务进行执行时序优化。系统中的每个节点从智能感知与预测算法获取任务到达序列、任务资源映射信息和节点状态预测信息，从系统状态检测中心获取节点资源信息和网络链路信息，并将以上信息输入基于多要素融合的算网资源智能调度算法。

基于多要素融合的算网资源智能调度算法包含任务簇间调度子模块以及任务簇内调度子模块两个模块。其中任务簇间调度子模块以及任务簇内调度子模块均部署于指挥节点上。各子模块的功能如下：

- **簇间任务调度子模块：**簇间任务调度子模块基于指挥节点对节点簇的资源监控情况进行适当的负载均衡，在指挥节点簇间进行任务调度。簇间任务调度输入所有节点簇的资源情况、任务队列情况、网络链路情况以及当前节点簇的任务资源需求信息，并输出当前节点簇任务调度决策。簇间任务调度子模块采用多节点联合决策，在各指挥节点间进行参数共享以实现决策的联合更新。

- **簇内任务调度子模块：**在检测到节点内剩余资源无法按时完成任务时，簇内任务调度子模块对节点簇内的节点进行负载均衡，动态调整节点间的任务分

配，优化簇内资源利用率。簇内任务调度子模块的输入包括任务资源需求、任务可调度状态、任务紧急程度及容忍时延、网络链路状态及节点内资源状态，并输出任务调度策略。簇内任务调度模块基于双延迟深度确定性策略梯度进行快速决策，减少通信开销，确保任务执行的效率和稳定性。

基于多要素融合的算网资源智能调度算法整体流程如图 2 所示：

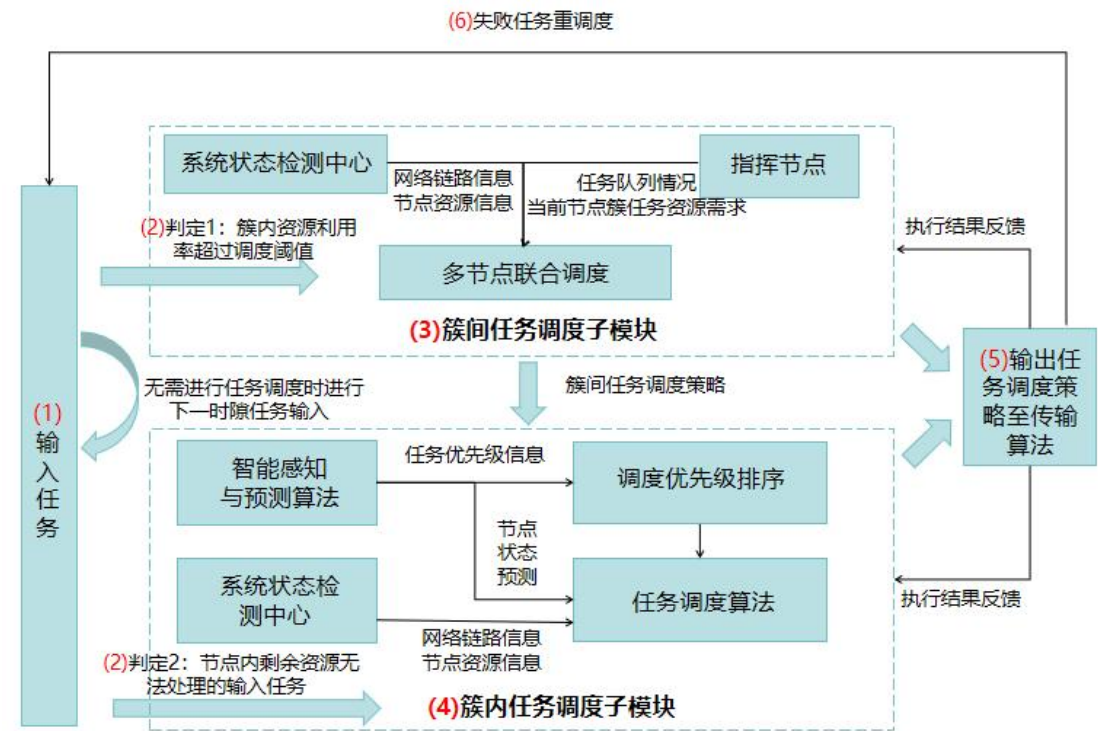


图 2 基于多要素融合的算网资源智能调度算法整体流程

(1) 对系统进行初始化，将节点以及节点簇进行编号。基于多要素融合的算网资源智能调度算法从智能感知与预测算法和系统状态检测中心获取输入以及到达任务，并对每个节点中产生的任务进行编号。

(2) 对各节点簇资源利用率以及各节点内任务资源需求情况进行检测。当节点簇资源利用率超过一定值、且高于所有簇平均资源利用率时，判断当前时隙需要进行簇间任务调度及簇内任务调度，并检测是否存在节点剩余资源无法满足任务资源需求的情况；当无需进行簇间任务调度，且检测到有节点内存在剩余资源无法满足任务资源需求的情况，判断当前时隙仅需要进行簇内任务调度。否则，当前时隙无需进行任务调度，无需执行（3）-（6）。

(3) 当需要进行簇间调度时，指挥节点监测出所需调度的任务资源需求以及所有节点簇的任务队列情况并从系统状态检测中心获取当前节点资源信息及网络链路信息，确定簇间调度策略。

(4) 当需要进行簇内调度时，从智能感知与预测算法获取优先级信息和当前节点簇内节点状态预测情况、从系统状态检测中心获取网络链路信息和节点资源信息。对需要调度的任务（包括簇间调度到节点簇的任务以及因节点无法处理任务产生的需调度任务）进行优先级排序并产生任务调度策略。

(5) 执行簇间及簇内任务调度策略。

(6) 在当前时隙任务执行完成后，检测任务的完成情况，将未完成的任务退回队列进行重新调度。

基于多要素融合的算网资源智能调度算法具体输入输出格式如表 1 所示：

表 1 输入数据

序号	数据项	数据来源	内容解释	数据类型
1	节点类型	系统状态检测中心	0: 发射节点 1: 指挥节点	int
2	节点簇内的指挥节点	系统状态检测中心	指挥节点标识符	int
3	节点簇包含的节点	系统状态检测中心	簇内节点标识符	array[int]
4	任务 CPU 资源需求	智能感知与预测算法	任务的 CPU 资源需求（单位：Flops）	int
5	任务 NPU 资源需求	智能感知与预测算法	任务的 NPU 资源需求（单位：Flops）	int
6	任务内存使用量	智能感知与预测算法	任务执行所需的内存使用量（单位：GB）	int
7	任务磁盘存储量	智能感知与预测算法	任务执行所需的磁盘存储量（单位：GB）	int
8	任务计算量	智能感知与预测算法	任务执行的总计算量（单位：Flop）	int
9	紧急程度	智能感知与预测算法	任务的紧急程度，越小表示任务越紧急	enum[1, 2, 3, 4, 5]
10	容忍时延	智能感知与预测算法	任务对应的容忍时延（单位：ms）	float32
11	NPU 资源剩余	智能感知与预测算法	节点内的剩余 NPU 资源量（单位：Flops）	int
12	CPU 资源剩余	智能感知与预测算法	节点内的剩余 CPU 资源量（单位：Flops）	int

13	内存资源剩余	智能感知与预测算法	节点内的剩余内存资源量（单位：GB）	int
14	磁盘资源剩余	智能感知与预测算法	节点内的剩余磁盘资源量（单位：GB）	int
15	节点静态功耗	系统状态检测中心	节点空闲状态下的静态功耗（单位：W）	float32
16	节点满载功耗	系统状态检测中心	节点满载状态下的最大功耗（单位：W）	float32
17	链路容量	系统状态检测中心	链路传输容量（单位：Mbps）	float32
18	实时链路速率	系统状态检测中心	当前链路传输速率（单位：Mbps）	float32
19	实时链路发射功率	系统状态检测中心	当前链路发射功率（单位：W）	float32

表 2 输出数据

序号	数据项	内容解释	数据格式
1	任务来源节点	产生任务的节点标识符	int
2	任务编号	任务标识符	int
3	任务执行节点	执行任务的节点标识符	int

3.2. 设计方案

3.2.1. 系统模型

本节对节点模型、网络模型、任务模型以及时延模型进行介绍。系统模型中的变量对应关系如表 3 所示：

表 3 系统模型变量符号及其含义

变量符号	含义描述
$\mathcal{N}$	所有节点的集合
ξ	所有节点簇的集合
$\mathcal{T}$	所有时隙的集合
$n = (\theta_n, f_n, m_n)$	节点 n 的属性（节点类型、计算能力和存储能力）
$\eta_{n_0} = (f_{\eta_{n_0}}, m_{\eta_{n_0}})$	节点簇属性（节点簇内的总计算能力和存储能力）
$R_j^i(t)$	在 t 时刻指挥节点与发射节点的对应关系

$\mathcal{W}_n(t)$	节点 n 在 t 时刻的等待队列
$\mathcal{C}_n(t)$	节点 n 在 t 时刻的计算队列
$\mathcal{W}_n^{in}(t)$	t 时隙新到达等待队列 $\mathcal{W}_n(t)$ 的所有任务存储量
$\mathcal{W}_n^{out}(t)$	t 时隙等待队列 $\mathcal{W}_n(t)$ 可调度的所有任务的存储量
$\mathcal{W}_n^{ove}(t)$	t 时隙等待队列 $\mathcal{W}_n(t)$ 由于超过等待队列长度限制而溢出的所有任务的存储量
m_n	节点 n 的最大存储能力
$\mathcal{C}_n^{in}(t)$	t 时隙新到达计算队列 $\mathcal{C}_n(t)$ 的所有任务的计算量
$\mathcal{C}_n^{out}(t)$	t 时隙计算队列 $\mathcal{C}_n(t)$ 正在执行的所有任务的计算量
$\mathcal{C}_n^{sur}(t)$	t 时隙计算队列 $\mathcal{C}_n(t)$ 中超出可容忍时延的所有任务的计算量
$\mathcal{W}_{\eta_{n_0}}(t)$	指挥节点 n_0 指挥的节点簇 η_{n_0} 的等待队列
$\mathcal{C}_{\eta_{n_0}}(t)$	指挥节点 n_0 指挥的节点簇 η_{n_0} 的计算队列
$m_{\eta_{n_0}}$	节点簇 η_{n_0} 任务队列的最大存储能力
$\mathcal{P}$	网络链路的集合
u_p	链路 p 的最大链路容量
$v_p(t)$	链路 p 的实时链路传输速率
$Pow_p(t)$	链路 p 的实时链路发射功率
$\mathcal{P}_{n,n'}$	出发节点 n 与目标节点 n' 之间的候选路径集合
$M_n(t)$	节点 n 在 t 时刻到达的所有任务集合
$m_{n,k}(t)$	节点 n 在 t 时刻到达的第 k 个任务
$m_{n,k}(t)$ = $(d_{n,k}(t), sto_{n,k}(t), \psi_{n,k}(t), c_{n,k}(t), \tau_{n,k}(t), z_{n,k}(t))$	任务 $m_{n,k}(t)$ 的指标需求（需要的内存使用量，磁盘存储量，亲和度性质，计算量，最大时间容忍限度以及任务的重要程度）
$E_{n,k,t}(t^*)$	任务 $m_{n,k}(t)$ 在时隙 t^* 的成功执行任务集合

$a_{n,k}(t^*)$	任务 $m_{n,k}(t^*)$ 的调度决策
$q_{n,k}(t)$	任务 $m_{n,k}(t)$ 的重要程度
$\tau_{n,k}(t)$	任务最大容忍时间
$y_{n,k,t}(t^*)$	任务 $m_{n,k}(t)$ 在 t^* 时刻的优先级
$D_{n,k,t^*}^{com}(t)$	任务在节点 $a_{n,k}(t^*)$ 上执行时的计算时延
$D_{n,k,t^*}^{wait}(t)$	任务在等待队列中与计算队列中等待执行的时延
$\rho_{n,k}(t^*)$	优先级高于任务 $m_{n,k}(t^*)$ 的任务集合
$D_{n,k,t^*}^{tran}(t^*)$	任务 $m_{n,k}(t^*)$ 所有数据的传输总时延
$D_{n,k,t^*}(t)$	任务 $m_{n,k}(t^*)$ 的总时延
$o(t)$	t 时隙的并行任务集合
$D(t)$	t 时隙并行任务 $o_{n,k,t^*}(t)$ 的并行总时延
$E_{n,k,t^*}^{tran}(t)$	任务 $m_{n,k}(t^*)$ 在 t 时隙的传输能量消耗
$E_{tran}(t)$	t 时隙的任务传输总能耗
$E_{n,k,t^*}^{cpu}(t)$	任务 $m_{n,k}(t^*)$ 在 t 时隙的计算能量消耗
$u_n^{n,k}(t)$	任务 $m_{n,k}(t^*)$ 的任务计算利用率
$E_{cpu}(t)$	t 时隙的任务计算总能耗
$E(t)$	t 时隙任务消耗的总能量

3.2.1.1. 节点模型

本系统使用 $\mathcal{N} = \{1, \dots, n, \dots, N\}$ 表示所有节点的集合， $\xi = \{1, \dots, \eta_n, \dots, \xi\}$ 表示所有节点簇的集合。定义时隙集合为 $\mathcal{T} = \{1, \dots, t, \dots, T\}$ 。

节点属性：系统中的各个节点分为指挥节点和发射节点两种类型，并搭载着异构的计算与存储资源，记节点 $n \in \mathcal{N}$ 的属性为 $\mathbf{n} = (\Theta_n, f_n, m_n)$ ，其中 Θ_n, f_n, m_n 分别代表节点 n 的节点类型（0和1分别表示发射节点与指挥节点）、计算能力和存储能力。

节点簇属性：将指挥节点 n_0 所指挥的节点簇表示为 η_{n_0} ，其节点簇属性表示为 $\eta_{n_0} = (f_{\eta_{n_0}}, m_{\eta_{n_0}})$ ，其中 $f_{\eta_{n_0}}$ 和 $m_{\eta_{n_0}}$ 分别表示节点簇内的总计算能力和存储能力，可表示为：

$$f_{\eta_{n_0}} = \sum_{n \in \eta_{n_0}} f_n$$

$$m_{\eta_{n_0}} = \sum_{n \in \eta_{n_0}} m_n$$

在 t 时刻指挥节点与发射节点的对应关系表示为 $R_j^i(t)$ ，其中 $R_j^i(t) = 1$ 表示在 t 时刻下指挥节点 i 可以直接指挥发射节点 j 。

队列更新：对于节点 n ，其上共有等待队列 $\mathcal{W}_n(t)$ 与计算队列 $\mathcal{C}_n(t)$ 两类队列，分别用于存放接入到该节点等待调度的任务和调度后到达该节点的任务。

在每个时隙 t ，根据任务到达与执行状态进行队列的更新。记 t 时隙新到达等待队列 $\mathcal{W}_n(t)$ 的所有任务的存储量为 $\mathcal{W}_n^{\text{in}}(t)$ ，该时隙可调度的所有任务的存储量为 $\mathcal{W}_n^{\text{out}}(t)$ ，由于超过等待队列长度限制而溢出的所有任务的存储量为 $\mathcal{W}_n^{\text{ove}}(t)$ ，则等待队列在下一时隙 $t+1$ 的更新方式可表示为：

$$\mathcal{W}_n(t+1) = \min\{\mathcal{W}_n(t) + \mathcal{W}_n^{\text{in}}(t) - \mathcal{W}_n^{\text{out}}(t) - \mathcal{W}_n^{\text{ove}}(t), m_n\}$$

其中 m_n 为节点 n 的最大存储能力。

记 t 时隙新到达计算队列 $\mathcal{C}_n(t)$ 的所有任务的计算量为 $\mathcal{C}_n^{\text{in}}(t)$ ，正在执行的所有任务的计算量为 $\mathcal{C}_n^{\text{out}}(t)$ ，队列中超出可容忍时延的所有任务的计算量为 $\mathcal{C}_n^{\text{sur}}(t)$ ，则计算队列在下一个时隙 $t+1$ 的更新方式如下：

$$\mathcal{C}_n(t+1) = \mathcal{C}_n(t) + \mathcal{C}_n^{\text{in}}(t) - \mathcal{C}_n^{\text{out}}(t) - \mathcal{C}_n^{\text{sur}}(t)$$

节点簇队列更新：指挥节点 n_0 所指挥的节点簇表示为 η_{n_0} ，其等待队列与计算队列分别为簇内所有节点的等待队列与计算队列的集合，表示为：

$$\mathcal{W}_{\eta_{n_0}}(t+1) = \{\mathcal{W}_{n_0}(t+1), \dots, \mathcal{W}_{n_L}(t+1)\}$$

$$\mathcal{C}_{\eta_{n_0}}(t+1) = \{\mathcal{C}_{n_0}(t+1), \dots, \mathcal{C}_{n_L}(t+1)\}$$

其中 $\eta_{n_0} = \{n_0, \dots, n_L\}$ 表示节点簇 η_{n_0} 内包含的节点集合。

3.2.1.2. 网络模型

使用 $\mathcal{P}=\{1, ..., p, ..., P\}$ 表示网络链路的集合。对于链路 $p \in \mathcal{P}$ ，记 u_p 为最大链路容量， $v_p(t)$ 为实时链路传输速率， $Pow_p(t)$ 为实时链路发射功率。指定 $\mathcal{P}_{n,n'} = \{\mathcal{P}_{n,n'}^1, ..., \mathcal{P}_{n,n'}^h, ..., \mathcal{P}_{n,n'}^H\}$ 为出发节点 $n \in N$ 与目标节点 $n' \in N$ 之间的 H 条候选路径，而 $\mathcal{P}_{n,n'}^h \in \mathcal{P}_{n,n'}$ 表示第 h 条路径。

3.2.1.3. 任务模型

记 $M_n(t) = \{m_{n,1}(t), ..., m_{n,k}(t), ..., m_{n,K}(t)\}$ 表示某节点 n 在 t 时刻到达并需要调度的所有任务集合。 $m_{n,k}(t)$ 表示节点 n 在 t 时刻到达的第 k 个任务。则对于节点簇 η_{n_0} ，其在 t 时刻到达并需要调度的所有任务集合为 $M_{\eta_{n_0}}(t) = \{M_n(t) | n \in \eta_{n_0}\}$ 。

以元组形式记录每个任务 $m_{n,k}(t) \in M_n(t)$ 的指标需求：

$$m_{n,k}(t) = (d_{n,k}(t), sto_{n,k}(t), \psi_{n,k}(t), c_{n,k}(t), \tau_{n,k}(t), z_{n,k}(t)),$$

其中 $d_{n,k}(t)$, $sto_{n,k}(t)$, $\psi_{n,k}(t)$, $c_{n,k}(t)$, $\tau_{n,k}(t)$, $z_{n,k}(t)$ 分别表示任务 $m_{n,k}(t)$ 需要的内存使用量，磁盘存储量，亲和度性质（CPU 或 NPU 占用程度），计算量，最大时间容忍限度以及任务的重要程度，其中任务的重要程度以 $[0,1]$ 区间的值确定。定义任务 $m_{n,k}(t)$ 在时隙 t^* 对于节点 n' 的可调度状态为 $I_{n,k,t}^{n'}(t^*) \in \{0,1\}$ ，当任务 $m_{n,k}(t)$ 能够在节点 n' 执行时， $I_{n,k,t}^{n'}(t^*) = 1$ ，反之为0。定义任务 $m_{n,k}(t)$ 在时隙 t^* 的执行成功情况为 $E_{n,k,t}(t^*) \in \{0,1\}$ ，当任务成功执行时 $E_{n,k,t}(t^*) = 1$ ，反之为0。

设任务 $m_{n,k}(t)$ 在时隙 t^* 对于节点簇 η_{n_0} 的可调度状态为 $I_{n,k,t}^{\eta_{n_0}}(t^*)$ ，当节点簇 η_{n_0} 内所有节点对任务 $m_{n,k}(t)$ 的可调度状态均为0时， $I_{n,k,t}^{\eta_{n_0}}(t^*) = 0$ ，反之为1。

3.2.1.4. 任务计算优先级模型

在现代化战场环境中，需要对计算任务从多个方面衡量其重要程度和紧急程度，以实现精确的计算优先级感知，从而在调度时保障高优先级任务的优先执

行。以任务 $m_{n,k}(t)$ 为例，其在时隙 t^* 的计算优先级取决于以下四个因素。

(1) 任务紧急程度：战场上的任务类型繁多，如情报解密及分析，对敌导弹拦截，常规战术火力打击，电子侦查，电子干扰，智能决策以及战略反击等任务。不同任务的重要程度不同，根据这些任务的类型来分析这些任务的失败代价，为每种类型的任务预设任务重要性量化值，以此来从任务类型方面区分任务的紧急程度，记任务 $m_{n,k}(t)$ 的紧急程度为 $q_{n,k}(t)$ 。紧急程度标记越小，代表任务越紧急。

(2) 时延敏感性：定义为任务最大容忍时间 $\tau_{n,k}(t)$ 的倒数。作战任务的容忍时延指示了该任务的时延敏感性。若作战任务的容忍时延越小，意味着该任务对时延的敏感度高，紧急程度大，可能很快就会错过其最佳执行窗口期甚至导致其失败。例如对敌弹道导弹拦截，若无法在最佳时期内进行拦截，就只能寄希望于末端拦截，难度大大提高，很大概率会导致失败。

(3) 相对执行速度：定义为任务计算量的倒数。作战任务对计算资源的需求越小，其相对执行速度越快，优先执行此类任务有助于在计算资源有限的环境下迅速释放计算资源，减少后续任务的排队时间，减少任务堆积。

(4) 任务时效程度：量化任务持续处于低优先级而造成持续未处理的情况严重程度，记为 $\frac{t^*-t}{\tau_{n,k}(t)}$ ，表示任务等待时间与任务最大容忍时间的比值，该值越接近于 1 表示任务更需要被紧迫执行。

对上述四项指标进行归一化，得到任务 $m_{n,k}(t)$ 在 t^* 时刻的优先级 $y_{n,k,t}(t^*)$ 为：

$$y_{n,k,t}(t^*) = \omega_1 S[\frac{1}{q_{n,k}(t)}] + \omega_2 S[\frac{1}{\tau_{n,k}(t)}] + \omega_3 S[\frac{1}{c_{n,k}(t)}] + \omega_4 S[\frac{t^*-t}{\tau_{n,k}(t)}],$$

其中， $\omega_1, \omega_2, \omega_3, \omega_4$ 为权重因子，且满足： $0 < \omega_1, \omega_2, \omega_3, \omega_4 \leq 1$ ， S 表示归一化操作。在完成计算优先级确定后，任务将按照优先级从高到低排序，等待调度到节点执行。

3.2.1.5. 时延模型

记当下调度时隙为 t^* ，任务 $m_{n,k}(t) \in M_n(t)$ 的调度决策为 $a_{n,k}(t^*)$ ，决策空间为系统中的所有节点，例如 $a_{n,k}(t^*) = n'$ 表示将任务 $m_{n,k}(t)$ 调度至节点 n' 上的计算队

列中。则可得到当下可调度任务的时延如下：

(1) 计算时延

当任务在节点 $a_{n,k}(t^*) = n'$ 上执行时，其计算时延包括等待时延与执行时延两部分，记为：

$$D_{n,k,t^*}^{\text{com}}(t) = \mathbb{I}\{a_{n,k}(t^*) = n'\} \cdot \frac{c_{n,k}(t)}{f_{n'}} + D_{n,k,t}^{\text{wait}}(t^*),$$

其中 $\mathbb{I}\{x\}$ 为条件判断函数，当 x 为真时 $\mathbb{I}\{x\} = 1$ ，反之为 0。 $c_{n,k}(t)$ 表示任务 $m_{n,k}(t)$ 的计算量， $f_{n'}$ 表示节点 n' 的计算能力， $\frac{c_{n,k}(t)}{f_{n'}}$ 表示任务在节点上执行的时延。 $D_{n,k,t}^{\text{wait}}(t^*)$ 表示截至时隙 t^* 任务在等待队列中与计算队列中等待执行的时延，可表示为：

$$D_{n,k,t}^{\text{wait}}(t^*) = \mathbb{I}\{a_{n,k}(t^*) = n'\} \cdot (t^* - t) + \sum_{m_{n',k'}(t') \in \rho_{n,k}(t^*)} D_{n',k'}^{\text{com}}(t'),$$

其中，第一项为任务在等待队列 $\mathcal{W}_n(t)$ 中的等待时延，第二项为任务在计算队列 $\mathcal{C}_n(t^*)$ 中的等待时延， $\rho_{n,k}(t^*)$ 表示在完成优先级排序后的计算队列中，优先级高于任务 $m_{n,k}(t)$ 的任务集合， $m_{n',k'}(t')$ 为该集合中的任务。

(2) 传输时延

在任务调度中会产生任务从任务产生节点 n 传输到执行节点 n' 的时延，记任务 $m_{n,k}(t^*)$ 的数据经路径 $\mathcal{P}_{n,n}^h$ 传输到节点 n' 的传输时延为：

$$D_{n,k,t^*}^{\text{tran}}(t) = \mathbb{I}\{a_{n,k}(t^*) = n'\} \cdot \text{sto}_{n,k}(t^*) \cdot \sum_{p \in \mathcal{P}_{n,n}^h} \frac{1}{v_p(t)},$$

综上，对于任务 $m_{n,k}(t^*)$ ，可得到其总时延为：

$$D_{n,k,t^*}(t) = D_{n,k,t^*}^{\text{com}}(t) + D_{n,k,t^*}^{\text{tran}}(t) \cdot \mathbb{I}\{a_{n,k}(t^*) = n' \neq n\}$$

考虑到在 t 时隙可能有多个任务并行执行，记为 $o(t) = \{m_{n_1,k_1}(t^*), m_{n_2,k_2}(t^*), \dots\}$ 。采用这些并行任务中的最大任务时延作为 t 时隙这些并行任务 $o(t)$ 的并行总时延，记为：

$$D(t) = \max_{m_{n,k}(t^*) \in o(t)} D_{n,k,t^*}(t)$$

3.2.1.6. 能量模型

对于任务 $m_{n,k}(t^*) \in M_n(t)$ ，设其在时隙 t 的调度决策为 $a_{n,k}(t^*)$ ，则可得到当前可调度任务的能耗如下：

(1) 传输能耗

在任务的调度传输过程中，依据网络链路情况会产生一定的能量损耗。当前任务经路径 $\mathcal{P}_{n,n}^h$ 传输到节点 n' 的传输消耗能量可以表示为：

$$E_{n,k,t^*}^{\text{tran}}(t) = \mathbb{I}\{a_{n,k}(t^*) = n'\} \cdot \text{Pow}_p(t) \cdot D_{n,k,t^*}^{\text{tran}}(t),$$

其中 $\text{Pow}_p(t)$ 为路径 $\mathcal{P}_{n,n}^h$ 的实时链路发射功率， $D_{n,k,t^*}^{\text{tran}}(t)$ 为任务 $m_{n,k}(t^*)$ 的传输时延。

则在时隙 t ，任务传输消耗的总能量可以表示为：

$$E_{\text{tran}}(t) = \sum_{m_{n,k}(t^*) \in \text{Co}(t)} E_{n,k,t^*}^{\text{tran}}(t)$$

(2) 计算能耗

任务计算会依据资源需求产生一定的能量消耗。当 $a_{n,k}(t^*) = n'$ ，任务 $m_{n,k}(t^*)$ 的计算能耗可以表示为：

$$E_{n,k,t^*}^{\text{cpu}}(t) = \mathbb{I}\{a_{n,k}(t^*) = n'\} \cdot [P_{n'}^{\text{emp}} + (P_{n'}^{\text{max}} - P_{n'}^{\text{emp}}) \cdot u_{n'}^{n,k}(t)] \cdot \frac{c_{n,k}(t)}{f_n},$$

其中 $P_{n'}^{\text{emp}}$ 为节点 n' 空闲状态下的静态功耗， $P_{n'}^{\text{max}}$ 为节点 n' 满载状态下的最大功耗， $u_{n'}^{n,k}(t)$ 为任务 $m_{n,k}(t^*)$ 在节点 n' 上执行任务的任务计算利用率，表示为任务执行所需计算资源与节点内总计算资源的比值：

$$u_{n'}^{n,k}(t) = \min\left(\frac{c_{n,k}(t)}{f_n}, 1\right)$$

则在时隙 t ，任务计算消耗的总能量可以表示为：

$$E_{\text{cpu}}(t) = \sum_{m_{n,k}(t^*) \in \text{Co}(t)} E_{n,k,t^*}^{\text{cpu}}(t)$$

综上，在时隙 t ，任务的总能耗可以表示为：

$$E(t) = E_{\text{cpu}}(t) + E_{\text{tran}}(t)$$

3.2.2. 基于深度强化学习的节点簇内集中式任务协同调度机制

3.2.2.1. 协同任务调度机制流程

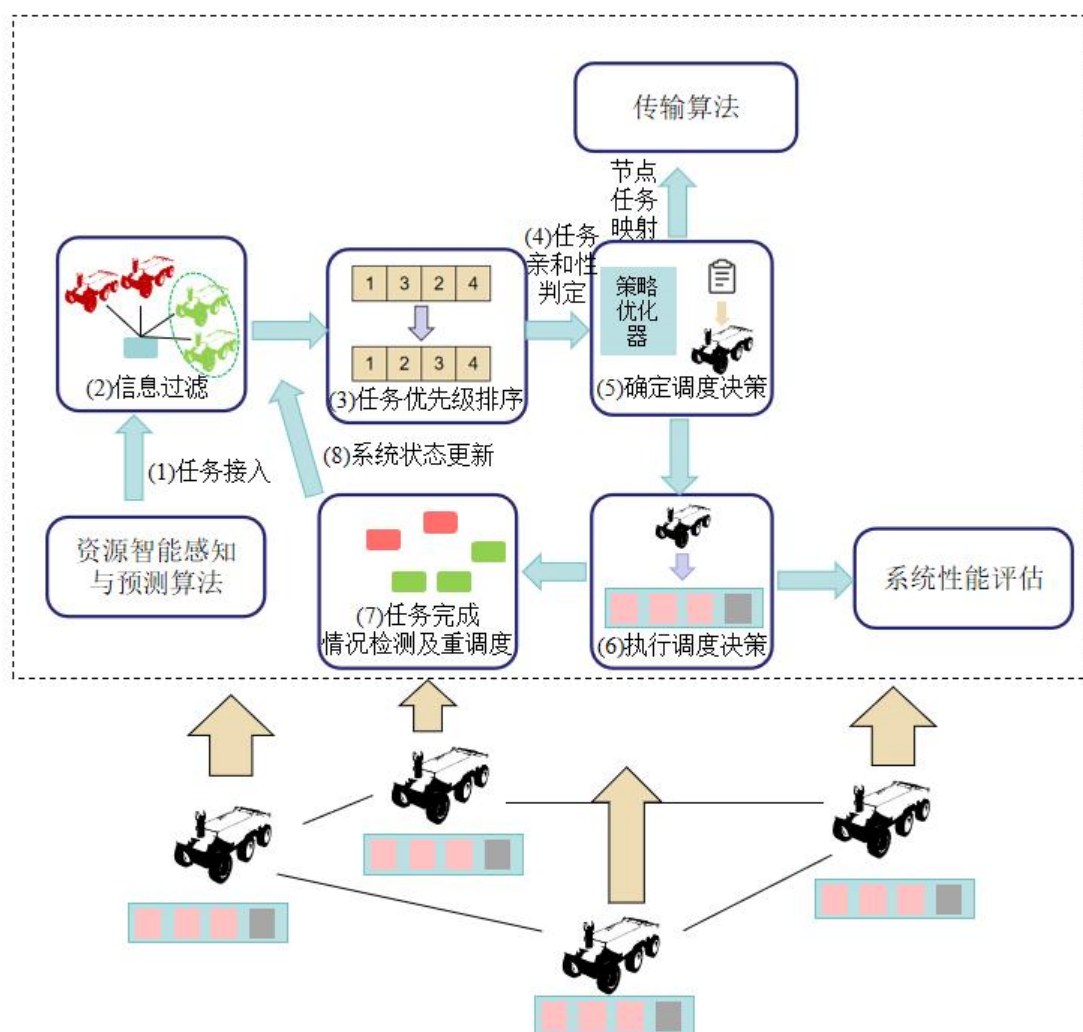


图3 节点间协同任务调度机制流程

如图3所示，在时隙 t ，节点之间的协同任务调度流程包含以下步骤：

(1) 任务接入：系统中的感知终端发起计算任务请求，请求下发到邻近的节点，加入到节点的等待队列中。获取当前需要调度的任务集合。

(2) 信息过滤：对于每个任务，系统根据任务需求从总节点序列中过滤不匹配或不可达的节点。

(3) 任务优先级排序：根据优先级排序机制确定计算队列中的任务优先级，并按优先级重新调整队列中的任务执行顺序。

(4) 任务亲和性判定：根据任务及节点的属性标记任务-节点亲和性情况。

(5) 确定调度决策：调度优化器为在该时隙可调度的任务队列制定调度策略，确定任务调度前往的计算节点。

(6) 执行调度决策：将可调度任务按照卸载决策加入到相应节点上的计算队列中。

(7) 任务完成情况检测及重调度：如果上一时隙有任务进行调度处理，检查任务执行状态，若任务执行失败且尚未超过其可容忍时延，则对该任务进行重调度，重新计算。

(8) 系统状态更新：在新的时隙到达之前，检测任务的执行状态与资源的使用状态，并对状态进行更新。

3.2.2.2. 节点-任务亲和性模型

在作战环境中，任务所需的异构资源不同，将主要资源情况较为相似的节点和任务匹配到一起可以提高资源利用率，减少节点内因单类资源不足无法执行其他任务的情况。基于以上情况，建立节点-任务亲和性模型作为调度算法的参考输入进行任务调度。

节点-任务亲和性是指将任务调度到具有特定标签或属性的节点上，以满足任务对资源或环境的特殊要求，例如目标识别等任务可以匹配到 NPU 资源相对丰富的节点上，数据备份等可以匹配到存储资源相对丰富的节点上，频繁通信需求的任务可以匹配到网络带宽较高的节点上。基于任务 $m_{n,k}(t^*)$ 的属性设置该任务亲和度标签为 $\psi_{n,k}(t) = [\psi_{type_{n,k}}(t), \psi_{percent_{n,k}}(t)]$ ，表示该任务主要需要的资源类型及其需求权重，通过 CPU 与 NPU 资源需求占总资源需求的比例计算，并保留其中占比较多的一种资源作为任务亲和度标签，例如当某任务所需的 CPU 和 NPU 资源比例分别为 60%和 40%时，该任务的亲和度标签为[CPU, 60%]。没有主要资源类型的任务则表示为 None，即所有节点均可适配。对于节点 n，其亲和度标签为 ψ_n ，表示该节点中剩余资源最多的资源类型及其资源剩余权重。

3.2.2.3. 马尔可夫决策过程

在战场环境中，计算任务、资源状态都具有极高的动态性，为了观察当下调度决策对系统未来状态变化的影响，我们研究长期的持续性调度过程。在每一个时隙，系统的状态仅与上一时隙的状态以及采取的调度决策相关，即具有马尔科夫性质，为此我们将长期的持续性调度过程构建为马尔可夫决策过程（Markov Decision Process, MDP）。在 MDP 中，我们主要关注状态、动作和奖励三个要素，具体如下：

(1) 状态：定义在 t 时刻的系统状态为所有节点上的资源使用状态、等待队列状态、计算队列状态、网络链路状态和任务状态（含任务优先级、任务亲和

性、任务可调度状态、任务资源权重需求)的集合,表示为:

$$\mathcal{S}(t) = \{\{s_n(t)|n \in \mathcal{N}\}, \{\mathcal{W}_n(t)|n \in \mathcal{N}\}, \{\mathcal{C}_n(t)|n \in \mathcal{N}\}, \\ \{L_p(t)|p \in \mathcal{P}\}, \{B_n(t)|n \in \mathcal{N}\}\}$$

其中 $s_n(t)$ 表示资源使用状态, $\mathcal{W}_n(t)$ 表示等待队列状态, $\mathcal{C}_n(t)$ 表示计算队列状态, $L_p(t)$ 表示网络链路状态, $B_n(t)$ 表示任务状态。

(2) **动作:** 对于任务 $m_{n,k}(t)$, 动作为其调度决策 $a_{n,k}(t)$, 即决定任务调度前往的计算节点的编号, 故对于所有在该时隙可调度的任务集合, 其动作集合为:

$$\mathcal{A}(t) = \{a_{n,k}(t)|m_{n,k}(t) \in M_n(t)\}$$

(3) **奖励:** 当执行动作后, 动作将与当前状态进行交互, 产生一定的奖励。考虑到系统对时延和能量的优化目标, 我们将奖励设定为以下三项, 以引导智能体更好地学习到较优的调度策略:

$$R(t) = -\gamma_1 D(t) - \gamma_2 E(t) + \gamma_3 Y(t)$$

其中, 第一项表示 t 时隙成功执行任务并行总时延的负值; 第二项表示 t 时隙所有成功执行任务消耗总能量的负值, 第三项表示优先级排序正确的奖励值, 确保算法优先调度优先级较高的任务, 表示为优先级之和, 即调度任务的优先级越高, 奖励函数也越大:

$$Y(t) = \sum_{m_{n,k}(t) \in M_n(t)} y_{n,k,t}(t^*)$$

在完成对调度过程的建立后, 为实现在系统中的任务调度, 需要寻找一条包含目标节点与任务传输路径的最大奖励值策略 π , 于是整个调度优化问题可以表述为:

$$Pl: \max_{\pi} \lim_{T \rightarrow \infty} \frac{1}{T} \sum_{t=1}^T E_{\pi} [R(t)]$$

3.2.2.4. 基于深度强化学习的任务调度算法

基于深度强化学习的任务调度算法在一轮训练内执行过程中的流程可以总结为以下4步:

(1) **环境交互:** 算法接收到新的计算任务队列, 在当前时刻 t 提取状态 $S(t)$,

其中包括节点信息、网络链路信息、计算任务的状态；

(2) 动作选择：深度强化学习中的 Actor 网络接收当前状态 $S'(t)$ ，输出调度动作 $A(t)$ 作为当前时隙的调度策略；

(3) 任务调度：环境根据 $A(t)$ 更新状态，模拟任务传输和执行，具体的流程包括任务经过计算出的路径传输到目标节点，在目标节点上执行任务优先级计算与计算单元匹配；

(4) 奖励计算：根据任务调度结果记录任务完成计算产生的时延，计算奖励值 $R(t)$ ，将状态转移信息 $(S(t), A(t), R(t), S(t+1))$ 存入经验回放池；

在完成以上流程后，首先从经验回放池中随机采样一批状态转移样本，通过目标网络估算下一状态的目标值，计算深度强化学习 Critic 网络的损失，并通过反向传播更新 Critic 网络的参数。每两次 Critic 更新后，优化 Actor 网络以提高调度策略的效果，同时采用软更新规则延迟更新目标网络的参数。训练过程中，实时监控平均任务时延和奖励值的变化趋势，并定期在测试任务流上评估策略性能。根据训练效果，动态调整超参数（如权重系数、更新速率、批量大小和学习率），并在动作选择中加入噪声以平衡探索与利用，逐步优化调度策略的表现。

在当前时隙需要调度的任务执行失败的情况下，需要依据资源情况对任务进行重调度，工作流程如下：

(1) 按照当前生成的调度策略将任务调度到各节点中。

(2) 执行一个单位时间后，将可并行的任务放到就绪队列中，检测当前核上每个任务执行的时间，如果时间等于该任务的预计执行时间则表明该任务已执行完毕，将其在对应的任务队列中删除。如果时间大于任务的预计执行时间，将该任务并重新放回队列中按照优先级等待调度。

(3) 再次调度当前可执行的任务，计算各任务可延迟的时间并重新设置优先级。

(4) 重复 (2) 和 (3) 的过程直到所有任务被调度完成。

算法的伪代码如下：

输入：系统状态 $\mathcal{S}(t)$ ，上一轮任务处理情况 $E(t)$

输出：新一轮调度策略 $\mathcal{A}(t)$

for $t = 1$ 直至 T 每轮训练：

for each $m_{n,k}(t) \in M_n(t)$:

if $E_{n,k,t}(t^*) = 0$:

```

    将任务 $m_{n,k}(t)$ 待处理任务状态集合 $B_n(t)$ 
  end if
end for
运行优先级任务调度算法，并生成策略 $\mathcal{A}(t)$ 
end for

```

3.2.3. 基于多节点联合调度的节点簇间调度机制

3.2.3.1. 节点簇间调度算法流程

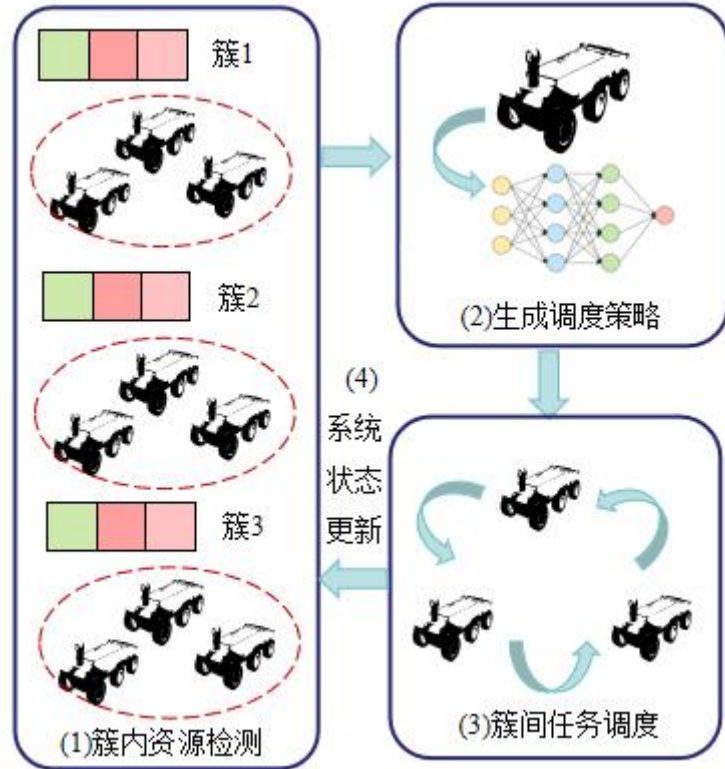


图 4 节点簇间联合调度算法流程

在时隙 t ，基于多节点联合调度的节点簇间调度机制流程具体流程如下，如图 4 所示：

（1）簇内资源检测：指挥节点对节点簇内资源利用率进行实时监控。当指挥节点监控的节点簇内资源利用率超过资源利用率阈值，且超过平均资源利用率的一定数值，启动节点簇间任务调度。

（2）生成调度策略：启动节点簇间任务调度的各指挥节点获取相邻节点簇的簇内资源利用率，并通过强化学习为任务和其余节点簇进行配对，生成簇内需调度任务的簇间调度策略。

（3）簇间任务调度：指挥节点输出簇间调度策略到传输算法，依据调度结果将任务发送到其余节点簇执行。

(4) 系统状态更新：在新的时隙到达之前，检测任务的执行状态与资源的使用状态，对状态进行更新。指挥节点对簇间调度模型的参数进行联合更新。

3.2.3.2. 任务簇间调度马尔可夫决策过程

在战场环境中，各节点簇中的资源状态都在不断动态变化。在每一个时隙，系统的状态仅与上一时隙的状态以及采取的调度决策相关，即具有马尔科夫性质，为此我们将长期的持续性调度过程构建为马尔可夫决策过程。在簇间任务调度的MDP中，我们主要关注状态、动作和奖励三个要素，单个指挥节点内簇间调度算法的MDP具体如下：

(1) **状态**：定义在 t 时刻的系统状态为相邻节点簇以及指挥节点所在节点簇的任务队列状态（包括队列中任务的任务优先级信息）以及剩余资源信息、网络链路信息和相邻节点簇以及指挥节点所在节点簇内任务信息（包含任务类型、任务紧急程度以及任务资源需求）的集合，表示为：

$$\mathcal{S}(t) = \left\{ \left\{ \mathcal{C}_{\eta_{n_0}}(t) \mid \eta_{n_0} \in \Gamma \right\}, \left\{ s_{\eta_{n_0}}(t) \mid \eta_{n_0} \in \Gamma \right\}, \left\{ L_{\eta_{n_0}}(t) \mid \eta_{n_0} \in \Gamma \right\}, \left\{ M_n(t) \mid n \in \eta_{n_0} \right\} \right\}$$

其中 Γ 为包含当前所有节点簇的集合， η_{n_0} 表示决策节点簇， $\mathcal{C}_{\eta_{n_0}}(t)$ 表示节点簇的任务队列状态， $s_{\eta_{n_0}}(t)$ 表示节点簇的剩余资源信息， $L_{\eta_{n_0}}(t)$ 表示网络链路状态， $M_n(t)$ 表示任务状态。

(2) **动作**：对于节点簇 η_{n_0} 中的节点 n ，需要调度的任务 $m_{n,k}(t)$ ，动作为其调度决策 $a_{n,k}(t)$ ，即决定任务调度前往的计算节点簇的对应编号，故对于所有在该时隙可调度的任务集合，其动作集合为：

$$\mathcal{A}_{\eta_{n_0}}(t) = \{a_{n,k}(t) \mid m_{n,k}(t) \in M_n(t), n \in \eta_{n_0}\}$$

(3) **奖励**：当执行动作后，动作将与当前状态进行交互，产生一定的奖励。考虑到系统对时延和能耗的优化目标，将奖励设定为簇间和簇内调度后邻近节点簇以及当前节点簇的平均奖励值，以引导智能体更好地学习到较优的调度策略：

$$R_{\eta_{n_0}}(t) = \frac{\sum_{\eta_n \in \Gamma} R_{\eta_n}(t)}{x}$$

其中， Γ 为包含当前指挥节点所在簇和所有邻近节点簇的集合， $R_{\eta_n}(t)$ 为节点簇 η_n 的任务调度奖励值（见簇内调度）， x 为 Γ 所包含的节点簇个数。

在完成对调度过程的建立后，为实现在系统中的任务调度，需要寻找一条包含目标节点与任务传输路径的最大奖励值策略 $\pi_{\eta_{n_0}}$ ，于是整个调度优化问题可以表述为：

$$Pl: \max_{\pi} \lim_{T \rightarrow \infty} \frac{1}{T} \sum_{t=1}^T E_{\pi_{\eta_{n_0}}} [R_{\eta_{n_0}}(t)]$$

3.2.3.3. 基于多智能体决策的簇间调度算法

基于强化学习的簇间任务调度算法在一轮训练内执行过程中的流程可以总结为以下 4 步：

（1）环境交互：算法接收到新的计算任务队列，包括节点簇的任务队列状态以及剩余资源信息、任务调度请求回复状态、网络链路信息和当前需要调度的任务信息；

（2）动作选择：深度强化学习中的 Actor 网络接收当前状态 $S(t)$ ，输出调度动作 $A(t)$ 作为当前时隙的调度策略；

（3）任务调度：环境根据 $A(t)$ 更新状态，模拟任务传输和执行，具体的流程包括任务经过计算出的路径传输到目标节点簇，并参与该节点簇内的调度；

（4）奖励计算：在整个调度流程完成后，返回任务处理结果到指挥节点，并根据各节点簇的任务调度结果计算奖励值 $R(t)$ ，将状态转移信息 $(S(t), A(t), R(t), S(t+1))$ 存入经验回放池；

在算法策略生成后，首先从经验回放池中随机采样一批状态转移样本，通过目标网络估算下一状态的目标值，计算深度强化学习 Critic 网络的损失，并通过反向传播更新 Critic 网络的参数。每两次 Critic 更新后，优化 Actor 网络以提高调度策略的效果，同时采用软更新规则延迟更新目标网络的参数。训练过程中，实时监控平均任务时延和奖励值的变化趋势，并定期在测试任务流上评估策略性能。根据训练效果，动态调整超参数（如权重系数、更新速率、批量大小和学习率），并在动作选择中加入噪声以平衡探索与利用，逐步优化调度策略的表现。同时，进行多智能体联合参数的更新。

3.3. 通用质量特性要求

- 实时性能：算法需要降低任务的总时延，能够在应对突发任务以及大算力任务时及时对任务进行调度处理，要求算法运行的时延不大于 5s。

- 泛化性能：算法需要处理不同节点产生的多种大算力任务，因此面对不同的任务，算法需要具有一定的泛化性能。在算法验证中，在 5 种不同场景下进行仿真并测算算法的奖励值，确保不同场景下算法奖励值稳定在目标值左右。
- 大规模调度性能：在实际应用中，为了处理大算力任务，所涉及的节点数量一般较大，因此算法需要在节点数量较大的情况下进行合理调度。在算法验证中，采用至少包含 1000 个节点的任务场景进行仿真并测算算法的奖励值。

3.4. 评估标准

为了验证算法的性能，算法的性能应基本优于其他对比算法，即 DQN, DDPG, DDQN, A3C 和 SAC。针对算法性能随不同变量变化的趋势图，目标仿真结果总结如下：

实时性能仿真验证：为了验证算法的实时性能，在所有仿真图中，预计算法执行时延小于 5s。任务到达量增加时，由于算法可以并行处理多个大算力任务，算法平均执行时延首先减小。当任务数量到达一定限度时，算法平均执行时延保持稳定。任务突发概率和并行任务数量变化时，由于算法对于突发任务处理具有实时性，算法平均执行时延应保持稳定。

泛化性能仿真验证：为了验证算法的泛化性能，在 6 种场景下进行算法测试，以模拟 6 种不同的任务场景。同时，使用任务到达量的变化测试不同场景下的稳定性。在任务到达量的变化下，由于算法的泛化性能，算法奖励值应趋近于稳定。

大规模调度性能仿真验证：为了验证算法的大规模调度性能，在仿真图中至少使用包含 1000 个分布式节点的场景进行验证。节点数量增加时，由于算力资源增多，在大算力任务数量较多的情况下，奖励值应呈增加趋势。

场景异常情况下的性能仿真验证：为了验证算法在场景异常情况下的性能，在仿真图中使用至少 30% 节点退网的场景进行验证。节点退网数量增加时，由于算力资源减少，任务完成速度变慢，奖励值呈减少趋势。

其中，泛化性能验证的场景设置如下：

（1）普通场景：考虑集群中节点位置保持固定，无节点产生干扰，过载，移动，退网等特殊情形下，保证计算任务最优化调度执行的情况。

（2）节点退网场景：考虑到作战集群中可能存在部分节点由于遭到打击产生损坏，导致其无法继续进行计算任务的执行，算法考虑在部分节点退网的环境

下最大程度保证未完成任务的调度与执行。在仿真中为每个节点设置退网概率，并在每个时隙根据退网概率判定节点是否退网以实现场景模拟。

(3) 节点干扰场景：考虑到作战集群中可能存在部分节点由于遭到电磁干扰产生信号损失，导致其通信能力以及计算处理能力产生下降，算法考虑在部分节点遭到干扰的环境下最大程度保证未完成任务的调度与执行。在仿真中为任务设置传输失败概率，并在任务需要传输时进行判定以实现场景模拟。

(4) 节点过载场景：考虑到作战集群中可能存在部分节点同时收到大量任务处理请求，导致其计算处理能力产生过载，任务处理产生卡顿的情况，算法考虑在部分节点计算能力过载的环境下最大程度保证未完成任务的调度与执行。在仿真中为节点设置突发任务概率，即每个节点概率收到计算能力过载的任务，并在每一个时隙对节点进行判定以实现场景模拟。

(5) 多优先级任务调度场景：考虑到产生的作战任务可能会产生较大的优先级差别，导致算法面对不同优先级的任务处理顺序不同。在仿真中为任务设置较大的优先级差距，并在每一个时隙产生不同优先级的任务实现场景模拟。

(6) 动态拓扑场景：考虑到现实作战环境中节点的拓扑网络时刻存在变化，导致算法需要应对不同的网络拓扑进行自适应调整。在仿真中设置节点拓扑分布以及网络链路的动态变化已实现场景模拟。

3.5. 验证方案

3.5.1. 验证要求

3.5.1.1. 验证指标

算法执行时延及任务执行时延应尽可能小，同时使得执行结果的奖励值尽可能高。在泛化性能验证下，算法表现效果均应符合评估标准。

参考设置：

- 算法执行时延：目标值可以设置为 5s 及以下。
- 任务执行时延：目标值可以设置为对比算法均值的 70%-90%。例如，如果与当前调度算法对比的其余算法任务执行时延为 D ，则调度算法的任务执行时延为 $70\%D-90\%D$ 。
- 任务奖励值：目标值可以设置为对比算法均值的 110%-130%。例如，如果与当前调度算法对比的其余算法任务奖励值为 R ，则调度算法的任务执行时延

为 110%R-130%R。

3.5.1.2. 验证方式

3.5.1.2.1. 仿真环境搭建

a) 平台架构：基于 Python 3.8 + PyTorch 1.12 构建分布式仿真平台，模拟 50 个移动节点（装甲车）的战场环境。

b) 动态场景：周期性注入突发任务与节点失效事件。

3.5.1.2.2. 数据输入规范

a) 训练集：80%数据通过 NS-3 网络仿真生成，涵盖多种场景。

b) 测试集：20%数据引入真实战场日志（脱敏处理），验证算法迁移能力。

3.5.1.2.3. 对比实验设计

a) 基线模型：DQN, DDPG, DDQN, A3C 和 SAC。

b) 评估流程：固定随机种子（seed=42），重复实验 5 次取均值，记录算法执行时延、任务执行时延及奖励值。

3.5.1.3. 验证条件

a) 深度学习框架：PyTorch 1.12，用于 TD3 网络实现及神经网络模型建模。

b) 多智能体学习框架：RLlib1.1+Ray2.1，用于支持分布式多智能体联合训练。

c) 辅助工具：NumPy1.21，Scikit-learn 1.2（数据预处理）。

3.5.2. 训练样本仿真生成

a) 战场仿真工具：基于 OMNeT++定制模块，模拟以下场景：

b) 电磁干扰：高斯白噪声模型（频段 2.4GHz，强度 0~120dB）。

c) 开源数据适配：对 Google Cluster Trace 2019 数据集进行 6 种战场化场景改造：

3.5.3. 验证项目

序号	试验项目	试验目的	试验内容
1	算法实时性	不同突发任务情况下算	实验中记录不同算法在相同规模测试集

序号	试验项目	试验目的	试验内容
	能验证	法执行时间少于 5s	上的运行时间，评估算法的计算效率。
2	算法泛化性能验证	6 种场景下算法执行的奖励值情况	实验中记录性能中心返回的算法奖励值数据，与对比算法结果比较，评估不同场景下算法的优越性
3	算法大规模调度性能验证	验证算法在 1000 个节点以上的场景的稳定性	实验中记录不同节点数量下的算法奖励值，并与对比算法比较，评估节点数量变化时算法表现的稳定性。
4	场景异常情况下的算法性能验证	验证算法在 30%节点异常退网时的稳定性	实验中记录不同节点退网比例状态下的算法奖励值，与对比算法结果比较，评估不同退网比例下算法的稳定性。

4. 技术指标满足情况分析

序号	技术指标	满足情况	技术支持
1	算法调度后的任务执行总时延	$\leq 5s$	通过簇间和簇内调度算法设计降低任务执行总时延
2	任务执行时延	$\leq$ 对比算法结果均值的 90%	通过簇间和簇内调度算法设计降低任务执行总时延
3	任务奖励值	$\geq$ 对比算法结果均值的 110%	通过簇间和簇内调度算法设计在降低任务执行总时延的同时，保持节点间负载均衡
4	算法执行时间	$\leq 5s$	通过强化学习模型及图神经网络模型的参数调整簇间和簇内调度算法的计算复杂度，在优化调度策略的同时降低调度算法执行时间

5. 研究进度、预期研究成果及应用方向

5.1. 研究进度

年度	主要研究内容	完成时间	完成形式
2024年	调研分析分布式算网资源调度的最新研究成果与大算力任务分割相关模型。	2024.12.30	形成调研报告
2025年	研究对比分布式算网资源调度算法并进行调研报告生成。	2025.010.30	形成调研报告
	调研主流任务调度算法,进行算法选型工作	2025.12.30	形成算法选型论证报告
	完成基于多要素融合的算网资源智能调度算法设计报告	2026.06.30	形成设计报告相关文档
	完成基于多要素融合的算网资源智能调度算法仿真代码编写	2026.09.30	试验验证、仿真
	完成基于多要素融合的算网资源智能调度算法仿真实例,并在实装上进行测试与验证	2026.10.30	试验验证、仿真、专家评审
	进行结项工作	2026.12.30	提交报告及实物类成果

5.2. 预期研究成果

1) 报告类成果

- 基于多要素融合的算网资源智能调度算法模型设计说明;
- 基于多要素融合的算网资源智能调度算法模型测试说明;
- 基于多要素融合的算网资源智能调度算法模型测试报告;
- 基于多要素融合的算网资源智能调度算法模型使用说明;
- 基于多要素融合的算网资源智能调度算法模型研制总结报告

2) 实物类成果

- 算法软件、训练模型及配套显示界面(含源代码+可执行程序);

5.3. 应用方向分析

5.3.1. 典型任务 1：时敏性任务

时敏任务对处理时延具有严格要求，任务到达模式呈现强突发性。传统调度方法缺乏对突发任务的动态响应能力，易导致任务处理延迟及任务超时。本算法通过节点簇间和簇内调度机制，有效提升系统对瞬时高负载的适应能力。在簇间层面，智能体基于全局负载信息进行任务分发决策，优先选择低时延、轻负载簇进行处理；在簇内层面，强化学习模型根据实时资源状态与任务情况，自主优化任务调度序列与资源分配策略。例如，在作战阶段转换导致识别任务激增时，系统可迅速将任务导向低负载簇，并通过簇内强化学习机制进一步提升任务响应速度与成功率。

5.3.2. 典型任务 2：优先级任务

在优先级任务调度过程中，对高优先级任务的优先响应成为算法的关键需求。传统的调度策略难以依据到达任务的优先级，对高优先级的任务进行优先响应调度，易出现后到达高优先级任务延后处理的情况。本算法基于多维度对任务优先级情况进行度量，针对度量后的任务优先级进行队列中高优先级任务的重新调度，将其调度到轻负载的节点中进行处理，以保证任务的优先完成。例如，当节点中到达高优先级任务、且当前节点任务队列较满，算法可将其调度到簇内或簇间的空队列节点进行任务处理，确保高优先级任务优先完成。

6. 保障条件及措施

6.1. 研发团队保障条件

项目研发团队由刘韵洁院士领衔，获中国电子学会创新团队奖，是北京邮电大学网络与交换技术全国重点实验室的核心力量。目前该团队已建立起了由院士、教授、副教授、讲师、博士后组成的层次分明的科研团队。团队在未来网络架构体系、能源互联网、云网融合与算力网络等方面具有深厚的研究基础，近 5 年来，在 IEEEJSAC、IEEETWC、ACMSigcomm 等国际顶级期刊会议累计发表 SCI/EI 论文 500 余篇，获授权发明专利 100 余项，国际授权 4 项，出版专著 15 部，软件著作权 50 余项。团队先后牵头承担了 1 项国家重大科技基础设施工程、1 项国家自然科学基金重大研究计划项目、3 项国家重点研发计划项目、5 项国家重点研发计划课题、1 项 973 项目、1 项国家 863 项目、2 项国家科技重大专项、10 项国家自然科学基金等 30 余项国家级项目以及 80 余项省部级与校企合作项目。成果先后获 2023 年中国通信学会科技进步特等奖、2023 年北京市自然科学

二等奖、2023 年中国通信学会科技进步一等奖、2017 年中国通信学会技术发明一等奖、2021 年中国航天科技集团科技进步二等奖，以及入选中国科协 2021 年“科创中国”先导技术榜单、2020 年世界互联网大会“世界互联网领先科技成果”、中国通信学会 2021 年未来网络领先创新科技成果等。

项目研发团队长期从事通信网、计算机网络体系结构及其关键技术研究，工作基础充分，科研条件完备，具有先进的仪器和较完善的通信实验仪器、开发平台，包括：高档工作站、服务器、高性能计算机、Cisco 路由器、HP 交换机、NetFPGA 板卡以及微型计算机、通信协议分析仪、传输测试仪、网络分析仪、误码测试仪等仪器；另外还有 NS3、Qualnet 等网络仿真软件，可为项目的顺利开展提供良好的工作支撑条件。此外，项目研发团队承担了信息与通信领域首项国家重大科技基础设施项目“未来网络试验设施（CENI，总投资 15.14 亿元）”的建设，已建设有覆盖全国 40 个城市、133 个边缘网络的基础设施，可支持一至七层各类技术的创新研究，可满足本项目等多种场景的试验验证需求。

6.2. 质量保障措施

研究团队将严格按照合同或计划任务书的要求完成项目目标任务。在项目前期，团队将充分进行调研和规划，准确获取项目的技术指标、需求及交付标准，并确保双方沟通清晰无误。调研过程中将进行多次确认，组织团队输出深入、完整且全面的调研报告与分析，确保项目规划准确无误并满足需求。

为确保项目交付质量，在整个项目执行过程中将实施严格的质量管理和测试机制，包括功能测试、性能测试和安全性测试等环节，同时按要求提交项目执行报告、成果总结报告及项目经费预算等相关材料，并参与定期检查或评估，确保项目全程透明、科学和高效。研究团队将遵守学术规范和职业道德，保证项目的科学性和创新性，确保输出结果的高质量与严谨性。

在项目交付后，将建立高效的沟通渠道，通过电话、邮件或其他约定的方式提供及时的技术支持与解答。同时，设立专门的技术支持团队，负责后续的技术服务和用户培训。该团队将及时收集反馈意见，处理突发技术问题，包括软件错误修复、故障排除以及产品性能优化等，以全面保障项目成果的稳定使用和持续改进。

6.3. 保密工作措施

在整个项目过程中，项目团队将严格遵守并执行相关的保密法规和协议，同时针对项目团队中学生参与的特殊情况制定详细管理方案。具体如下：

6.3.1. 内部任务分解与人员分工

坚持“涉密工作由涉密人员完成，非涉密学生仅参与非涉密环节”的总体原则，对项目任务进行分解和标识。按照“按需知密、最小授权”原则，严格限定项目成员可接触的信息范围和权限等级。对所有任务在立项阶段进行保密属性评估，明确标注为“涉密任务”“非涉密任务”或“边界任务”（可能涉及保密内容的接口任务），并建立任务清单和责任台账。

（1）涉密人员工作内容

涉密人员为已完成涉密审查并具备相应保密资格的科研人员和管理人员，其主要承担以下工作：

- 涉密方案与核心技术研发：包括项目总体方案设计、核心算法与关键技术研究、涉密技术路线论证及决策等。
- 涉密数据采集与处理：承担涉密原始数据的采集、预处理、分析建模和结果研判，确保涉密数据不向非授权人员扩散。
- 涉密文件编制与审定：负责撰写、编辑和审定涉密技术报告、设计文档、内部评估材料及对上级报送的涉密材料，对内容进行密级标注和敏感点把关。

（2）非涉密学生工作内容

非涉密学生不纳入涉密人员管理，其参与范围严格限定在经筛选的非涉密任务中，主要包括：

- 一般性实验与仿真：在提供的脱敏或虚拟样本数据、公开数据集基础上，开展算法验证、参数调优、对比实验等工作，不接触真实涉密数据。
- 辅助性分析与整理：协助涉密人员对经脱密处理后的统计结果进行整理、可视化展示和一般性规律分析，避免涉及数据来源、关键指标阈值等敏感内容。
- 非涉密文献调研与综述：围绕公开领域的基础理论、通用技术路线、公开标准和行业发展现状进行检索、阅读和撰写综述，不涉及项目内部技术细节和专有方法。

6.3.2. 岗前培训与签署保密责任书

对所有参与项目的学生和工作人员，均需进行入职及项目相关的保密培训，确保其充分了解保密规定和要求。培训内容包括保密法规、项目相关的文件管理规定、访问权限规则等。所有团队成员均需签署保密协议，与学生单独签订《非涉密参与保证书》，并对协议履行情况设立监督机制。

6.3.3. 涉密场所与办公区域管理

（1）区域划分与物理隔离：项目办公区域实行“三区管理”：涉密区、受控区和普通办公区。涉密区实行封闭管理，受控区为涉密前后过渡区域，普通区为学生及非涉密人员主要活动区域。涉密工作（含涉密文件阅读、涉密讨论、涉密设备操作）必须在涉密区内进行，禁止在开放实验室、公共自习区域或外部场所开展。通过门禁系统、视频监控和区域标识，对涉密区进行物理隔离，未经授权人员不得进入。

（2）人员出入与行为规范：涉密区实行门禁刷卡和访客登记制度，学生一般不得进入涉密区，确因工作需要进入受控区的，须由涉密人员全程陪同并记录理由和时间。涉密区内严禁携带个人手机、平板、U 盘等可记录、存储或无线通信的电子设备，确需使用的设备须通过保密审查和登记。在涉密和受控区域内，禁止随意谈论项目核心技术内容，会议和讨论需在指定会议室进行，按规定做好会议记录和资料回收。

6.3.4. 涉密文件与资料管理流程

（1）涉密文件全生命周期管理

涉密文件由涉密人员编制，按照内容敏感程度标注密级和保密期限，并填写《涉密文件登记表》，并统一由项目保密秘书进行编号、登记和建账，记录文件名称、编号、密级、份数、保管人和使用范围等信息。纸质涉密文件存放于专用保密柜内，实行双人双锁管理；电子文件存储在涉密服务器或加密硬盘中，通过严格账号和权限控制。

（2）涉密文件使用与流转控制

涉密文件仅限授权涉密人员在涉密场所内查阅，严禁外借、拍照、复印和私自携带出涉密区域。确需在项目内部有限流转时，由保密秘书办理传阅手续，对传阅人、时间和范围进行登记，并在规定期限内收回。学生不得接触任何标注为

涉密的文件，涉密内容如需间接体现给学生，必须由涉密人员进行脱敏、归纳后以非涉密形式提供。

(3) 电子文件管理与打印控制

部署物理与逻辑隔离的涉密网络系统，与非涉密办公网络严格分离，涉密网络仅供涉密人员使用，学生账号不得接入。同时对涉密电子文件采用强加密算法进行保护，并设置访问日志和操作审计功能，防止非法复制和外传。涉密文件仅允许在指定的保密打印设备上打印，打印任务须由涉密人员操作，系统自动记录打印人、时间和份数，打印后的废页、样张按涉密文件处理。

(4) 涉密文档销毁

对失效、作废或经审批同意销毁的涉密文件，统一使用专业保密文档销毁机进行物理粉碎，做到不可恢复。销毁过程由两名以上涉密人员现场见证，并填写《涉密文件销毁记录表》，列明销毁文件的种类、编号、密级和数量。对存储涉密信息的电子介质，在报废前按单位规定执行数据清除或物理销毁操作。

6.3.5. 保密检查与应急预案

(1) 定期检查与风险排查

建立定期保密检查机制，由项目负责人牵头、保密秘书及相关涉密人员参与，对人员管理、场所管理、文件管理和设备使用进行全面排查。重点检查学生在执行非涉密任务时是否存在违规存储、违规拷贝、跨区域传输等行为，并对发现的问题及时整改。检查情况形成书面报告，作为项目阶段性评估和总结的重要内容。

(2) 突发事件处置与责任追究

一旦发现可能存在泄密隐患或违规操作行为，立即启动项目保密应急预案：包括紧急断网、封存设备、停止相关任务、召回资料等措施。对已发生的泄密或严重违规行为，及时上报单位保密部门，配合开展调查，查明原因和影响范围，按规定追究相关责任人的法律和纪律责任。根据事件教训及时修订和完善相关管理制度、技术措施和培训内容，防止类似问题再次发生。

7. 承担研究任务主要人员概况

姓 名	职务/职称	单 位	参与周期	任务分工
-----	-------	-----	------	------

谢人超	教授	北京邮电大学	任务全周期	项目总负责人，全面负责研究整体规划、技术路线、科研统筹管理、理论指导等。
黄韬	教授	教授	任务全周期	协助项目组织实施，负责为算法理论研究提供技术支持
唐琴琴	副研究员	北京邮电大学	任务全周期	协助项目组织实施，负责为算法优化模型提供技术支持
冯立	博士	北京邮电大学	任务全周期	负责算法选型调研与分析
冯瑞琪	博士	北京邮电大学	任务前期	负责算法选型调研与分析
徐宏亮	博士	北京邮电大学	任务前期	数据采集与处理，完成任务负载预测模型训练与数据预处理模块开发。
许德昀	硕士	北京邮电大学	任务前期与中期	算法模块设计与研发
邓喆	硕士	北京邮电大学	任务前期与中期	算法模块设计与研发
周弘烨	硕士	北京邮电大学	任务前期与中期	算法模块设计与研发
张语嫣	硕士	北京邮电大学	任务中期	实现仿真实验环境搭建，优化实验数据记录与问题反馈机制。
万卓然	硕士	北京邮电大学	任务中期	数据实验结果可视化，完成报告撰写与模型结果核验。
李敏宽	硕士	北京邮电大学	任务中期与后期	协助完成数据分析与实验验证，辅助模型调优和实验结果记录。
李妃	硕士	北京邮电大学	任务后期	负责日志记录系统开发，完成实验问题的在线诊断和维护支持。

8. 技术风险分析

(1) 深度强化学习算法的复杂性与稳定性挑战

基于双延迟深度确定性策略梯度（TD3）的算网资源调度算法，虽然在处理连续动作空间和复杂决策问题上具有优势，但其算法本身的复杂性较高。算法的训练和收敛需要大量的计算资源和时间，且在实际应用中，算法的稳定性可能受到环境噪声、模型参数设置等因素的影响。此外，TD3 算法在处理大规模节点和

任务时，可能会出现训练效率下降、策略更新不稳定等问题，需要进一步优化算法结构和参数调整策略，以确保算法在复杂作战环境下的稳定性和收敛性。

(2) 任务优先级模型的动态适应性问题

任务优先级模型在确定任务执行顺序时，需要综合考虑任务的重要程度、时延敏感性、相对执行速度和任务时效程度等多个因素。然而，在实际作战环境中，这些因素可能会随时间动态变化，导致任务优先级模型的预测能力逐渐衰减。此外，任务优先级模型的参数设置和权重分配也会影响其预测性能，需要根据实际需求进行不断调整和优化。因此，需要设计更灵活的任务优先级模型，以适应动态变化的作战环境。

(3) 算网资源协同调度的实时性与准确性

算网资源的协同调度需要实时感知节点的资源状态和任务需求，以实现资源的合理分配和高效利用。然而，在实际作战环境中，节点的资源状态和任务需求可能会快速变化，导致调度决策的实时性和准确性受到挑战。此外，算网资源的异构性和分布性也会增加协同调度的复杂性，需要设计更高效的资源协同机制，以应对异构资源的动态变化。因此，需要进一步优化算网资源协同调度算法，提高其实时性和准确性。

(4) 多目标性能指标的权衡与优化

在算网资源调度中，需要同时考虑多个性能指标，如任务完成时间、资源利用率、系统吞吐量等。这些指标之间可能存在冲突，例如提高任务完成时间可能会降低资源利用率。因此，在设计调度算法时，需要在多个性能指标之间进行权衡和优化。此外，还需要考虑算法的复杂度和计算开销，以确保算法在实际应用中的可行性和效率。

9. 经费概算

本项目经费概算如下表所示。具体费用包括设备费、材料费、业务费、劳务费、差旅费以及知识产权及成果推广费。

序号	条款名称	金额（万元）
1	科学基金资助项目直接费用合计	50.00
2	设备费（研发所需设备购置与租赁）	10.00
3	材料费（耗材、研究所需试剂购买等）	8.00
4	业务费（调研、实验测试及第三方协作）	12.00
5	劳务费（参与人员补助、博士后津贴等）	15.00

6	差旅费（会议、课题调研）	5.00
---	--------------	------